

```

    customLaunchers: {
      ChromeHeadlessNoSandbox: {
        base: 'ChromeHeadless',
        flags: ['--no-sandbox']
      }
    },
    autoWatch: true,
    singleRun: true,
    browsers: ['ChromeHeadlessNoSandbox'],

    customLaunchers: {

```

7. Configure *Package.json* with scripts:

```

"scripts": {
  "ng": "ng",
  "start": "ng serve",
  "build": "ng build",
  "test": "ng test --code-coverage",
  "lint": "ng lint",
  "e2e": "ng e2e",
  "postinstall": "webdriver-manager update --standalone
false --gecko false"
},

"devDependencies": {
.
.
  "karma-junit-reporter": "2.0.1",
.
.
}

```

8. Install *Cordova Build Extension* in the organisation so that the pipeline can use this extension.
9. Go to *Pipelines* section and create a new pipeline. Connect to *Azure Repos Git*. Select a repository, and then select a template to build a pipeline from the start.
10. The following is the pipeline for the Ionic Cordova app:

```

trigger:
- master
pool:
  vmImage: 'vs2017-win2016'

stages:
- stage: "ContinuousIntegration"
  jobs:
  - job: "CI"
    steps:
    - script: |
      npm install

```

```

      npm install karma-junit-reporter --save-dev
      npm audit fix
      displayName: 'npm install and build'

# Perform Lint analysis
- script: |
  npm run lint > LintResults.txt
  displayName: 'Lint Analysis'

- task: CopyFiles@2
  inputs:
    Contents: 'LintResults.txt'
    TargetFolder: '$(build.artifactstagingdirectory)'

- task: PublishBuildArtifacts@1
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'
    publishLocation: 'Container'

# Execute unit tests
- script: |
  npm run test
  displayName: 'Unit Tests'

# Publish Junit Test Format results
- task: PublishTestResults@2
  inputs:
    testResultsFormat: 'JUnit'
    testResultsFiles: '**/TEST*.xml'

# Publish Code coverage details on Azure DevOps Dashboard
- task: PublishCodeCoverageResults@1
  inputs:
    codeCoverageTool: 'Cobertura'
    summaryFileLocation: '**/*coverage.xml'

# Quality verification of the build based on the line
coverage by Unit tests
- task: BuildQualityChecks@6
  inputs:
    checkCoverage: true
    coverageFailOption: 'fixed'
    coverageType: 'lines'
    coverageThreshold: '20'

# Create Empty Directory - Ref: https://stackoverflow.com/
questions/21276294/cordova-current-working-directory-is-not-
a-cordova-based-project
- task: PowerShell@2
  inputs:
    targetType: 'inline'

```